

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

TECHNICAL UNIVERSITY OF MOLDOVA

FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
DEPARTMENT OF SOFTWARE AND AUTOMATION ENGINEERING

Report

Laboratory Work No. 7

Process Lifetime Orchestration, Standard Stream Redirection, and Inter-Process Communication Pipelines in Linux

Elaborated by:
st. gr. FAF-241
Tihon Aurelian-Mihai

Verified by:
univ. assist.
Reiţman Patricia

Chişinău – 2026

Contents

INTRODUCTION **3**

1 THEORETICAL PROCESS MANAGEMENT AND STREAM PIPELINES **3**

1.1 Process creation, address space duplication, and program replacement mechanics 4

1.2 Process state transitions, synchronization, and zombie reclamation 5

1.3 File descriptor tables, stream redirection, and pipe flow control 5

1.4 Theoretical complexity comparison 6

2 EMPIRICAL VERIFICATION AND OPERATING SYSTEM ANALYSIS **6**

2.1 Process creation and execution with run.c 7

2.2 Zombie process creation, observation, and kernel reaping 8

2.3 Standard stream redirection via deterministic descriptor allocation 9

2.4 Two-stage inter-process pipeline construction and deadlock dynamics 9

2.5 Three-stage pipeline extension with multiple concurrent pipes 10

CONCLUSIONS **12**

REFERENCES **13**

ANNEX A **14**

ANNEX B **16**

INTRODUCTION

The execution model of modern Unix-like operating systems relies fundamentally on the separation of process creation from binary image execution. Unlike operating system environments that combine process initialization and binary invocation into a single monolithic primitive, Unix separates this lifecycle into two discrete system calls: `fork()`, which duplicates the calling process address space, and `execve()`, which replaces the process memory image with an entirely new executable. Synchronization between parent and child processes is established via the `wait()` family of system calls, which allows parent tasks to collect termination metrics, manage concurrency, and prevent resource leaks.

Equally fundamental to the Unix philosophy is the abstraction of inter-process communication and I/O streams through integer file descriptors. Because file descriptors are preserved across `fork()` and `exec()` operations, processes can manipulate standard input, standard output, and standard error before executing a new binary image. This architectural property enables the construction of complex software pipelines through unidirectional kernel buffers known as pipes. Programs remain completely agnostic regarding the physical destination or origin of their data streams, achieving modularity and composability.

The objective of this laboratory work is to investigate the mechanics of process creation, address space substitution, zombie process lifecycles, file descriptor reassignment, and inter-process communication pipelines under the Linux kernel. By implementing dedicated C programs without high-level scripting wrappers, the assignment examines how the shell coordinates command execution, handles standard stream redirection, and connects concurrent processes via unidirectional pipelines. All source code, build scripts, and automated test harnesses are maintained in the version control repository hosted on GitHub at `https://github.com/aurelian-th/os-2026-TihonAurelian`.

1 THEORETICAL PROCESS MANAGEMENT AND STREAM PIPELINES

1.1 Process creation, address space duplication, and program replacement mechanics

Process creation in the Linux operating system is mediated by the `fork()` system call, which creates an exact duplicate of the invoking process. As formalized in classical systems literature [1], [2], the newly created child process receives a unique process identifier (PID), its own task descriptor (`struct task_struct`), and an independent set of virtual memory structures. Historically, Unix implementations duplicated all physical memory frames during process creation. Contemporary Linux kernels eliminate this overhead by employing Copy-on-Write (COW) page table semantics.

Under Copy-on-Write, the parent process page tables are copied, but both parent and child page table entries point to the same physical memory frames. All shared frames are marked read-only in the Memory Management Unit (MMU). When either process attempts a write operation, the MMU triggers a write protection page fault. The kernel page fault handler intercepts this exception, allocates a new physical frame, copies the 4096-byte page contents, updates the faulting process page table entry with write permissions, and flushes the Translation Lookaside Buffer (TLB). The theoretical memory allocation overhead M_{alloc} associated with process cloning is governed by:

$$M_{\text{alloc}} = S_{\text{task_struct}} + S_{\text{ptables}} + \sum_{i=1}^{P_{\text{modified}}} S_{\text{page}} \tag{1.1}$$

where $S_{\text{task_struct}}$ denotes the kernel task descriptor size, S_{ptables} represents the architectural page table allocation, P_{modified} reflects the number of dirty pages modified post-fork, and S_{page} denotes the architectural page size (4096 bytes).

Once the child process is instantiated, it transitions to a new executable image by invoking the `execve()` system call (or C library frontends such as `execlp()`). The `execve()` routine releases the existing user address space, unmaps memory segments, loads the text and data segments from the target Executable and Linkable Format (ELF) binary, establishes a new user stack containing command line arguments (`argv`) and environment variables (`environ`), and resets the program counter to the ELF entry point (`_start`). Crucially, if `execve()` succeeds, it never returns to the calling code because the original text segment has been completely overwritten. Control returns to the caller only if an error occurs, such as file absence (*ENOENT*) or permission denial (*EACCES*).

1.2 Process state transitions, synchronization, and zombie reclamation

During its lifecycle, a process transitions through a finite set of operating states managed by the kernel scheduler: running or ready (`TASK_RUNNING`), interruptible sleep (`TASK_INTERRUPTIBLE`), uninterruptible sleep (`TASK_UNINTERRUPTIBLE`), stopped (`TASK_STOPPED`), and zombie (`EXIT_ZOMBIE`). When a process completes its execution, it invokes the `exit()` system call, transmitting an integer termination status to the kernel.

Upon process termination, the kernel immediately deallocates the process address space, page tables, and active file descriptors. However, the process descriptor (`struct task_struct`) cannot be released immediately. The process enters the zombie state, retaining its PID, termination exit status, and resource usage metrics in the global process table. This state persists until the parent process synchronizes using the `wait()` or `waitpid()` system call:

$$S_{\text{exit}} = \text{WEXITSTATUS}(status) = (status \gg 8) \wedge 0\text{xFF} \tag{1.2}$$

When the parent invokes `waitpid()`, the kernel extracts the termination status, releases the zombie task descriptor, and marks the PID as available for reallocation. If a parent process terminates without waiting for its children, the orphaned processes are automatically adopted by the initial userland supervisor process (`systemd` or `init`, PID 1), which continuously executes `wait()` to reclaim defunct descendants.

1.3 File descriptor tables, stream redirection, and pipe flow control

The Linux Virtual File System (VFS) maintains three primary descriptor abstractions: the per-process file descriptor table, the system-wide open file table, and the vnode/inode table. Each entry in the per-process file descriptor table contains flags (such as `FD_CLOEXEC`) and a pointer to an open file description entry in the system file table, which records the current file offset, access mode, and inode reference.

By convention, every standard process initializes with three standard streams: descriptor 0 bound to standard input (`stdin`), descriptor 1 bound to standard output (`stdout`), and descriptor 2 bound to standard error (`stderr`). POSIX descriptor allocation rules mandate that when an existing descriptor is released via `close()`, any subsequent allocation call (such as `open()` or `dup()`) deterministically assigns the lowest-numbered available index in the descriptor table:

$$\text{fd}_{\text{new}} = \min\{i \in \mathbb{N}_0 \mid \text{Table}[i] = \text{NULL}\} \tag{1.3}$$

Stream redirection operates by releasing descriptor 1 or 2 and opening a destination file, causing the new file description to bind to the standard output or error slot before calling `execve()`.

An inter-process communication pipe is constructed via the `pipe()` system call, which creates a unidirectional data channel managed in kernel memory. The call returns two file descriptors: $p[0]$ for reading and $p[1]$ for writing. The pipe buffer is implemented as a circular memory ring buffer of fixed capacity (typically 65,536 bytes on modern Linux kernels). Read operations on an empty pipe block the calling process until data is written, while write operations on a full pipe block until buffer space is reclaimed. Crucially, a reading process receives an End-of-File (EOF) indication (returning zero bytes read) only when all open write descriptors referencing the pipe across all processes in the system have been closed.

1.4 Theoretical complexity comparison

The performance characteristics of process lifecycle operations and stream redirection primitives are dictated by kernel memory management and context switching costs. Table 1.1 summarizes the asymptotic time and space complexities across standard process and pipeline operations [3], [4].

Table 1.1 – Asymptotic time and space complexities of process and pipeline primitives

Kernel Operation	**Time (Average)**	**Time (Worst)**	**Space Complexity**
Process Fork (`fork`)	$O(P_{\text{tables}})$	$O(P_{\text{tables}})$	$O(P_{\text{tables}})$
Program Overlay (`execve`)	$O(S_{\text{binary}})$	$O(S_{\text{binary}})$	$O(S_{\text{binary}})$
Child Synchronization (`waitpid`)	$O(1)$	$O(C_{\text{children}})$	$O(1)$
Descriptor Duplication (`dup2`)	$O(1)$	$O(1)$	$O(1)$
Pipe Buffer Stream Read/Write	$O(B_{\text{trans}})$	$O(B_{\text{trans}})$	$O(1)$
K-Stage Concurrent Pipeline	$O(K+\sum B)$	$O(K+\sum B)$	$O(K \cdot S_{\text{pipe}})$

In Table 1.1, parameter P_{tables} denotes the number of page table pages copied during initial process instantiation, S_{binary} represents the physical byte size of the executable ELF image loaded into memory, C_{children} reflects the child process list traversal length, B_{trans} denotes the number of bytes transferred across the pipe buffer, and K corresponds to the pipeline stage count with individual buffer capacity S_{pipe}.

2 EMPIRICAL VERIFICATION AND OPERATING SYSTEM ANALYSIS

2.1 Process creation and execution with run.c

The practical investigation commenced by analyzing the canonical three-step execution workflow implemented by command line shells. The program `run.c` was constructed to emulate the execution of `ls -l`. The parent process outputs its process identifier, invokes `fork()`, and evaluates the returned value. In the child branch (PID $==0$), the process outputs its assigned PID and parent PID using `getpid()` and `getppid()`, and subsequently executes `execlp("ls", "ls", "-l", NULL)`. The parent process invokes `waitpid()` to synchronize with child termination.

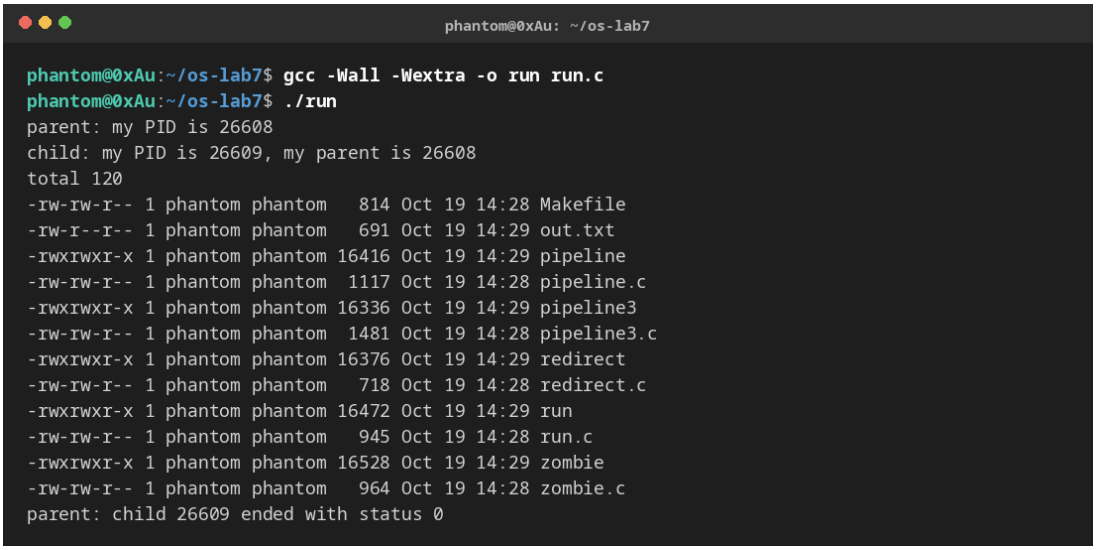


Figure 2.1 – Process creation and execution workflow with run.c

According to Figure 2.1, the parent process was assigned PID 26608, while the newly created child process received PID 26609. The child confirmed that its parent PID matched 26608. Upon executing `execlp()`, the child address space was replaced by the binary image of `ls`, which printed the detailed directory listing. Once the listing finished, the child exited with status 0, which was harvested by `waitpid()` in the parent.

To evaluate program replacement mechanics under failure conditions, the executable target was modified to execute `date` and an invalid program name `nosuchprog`.

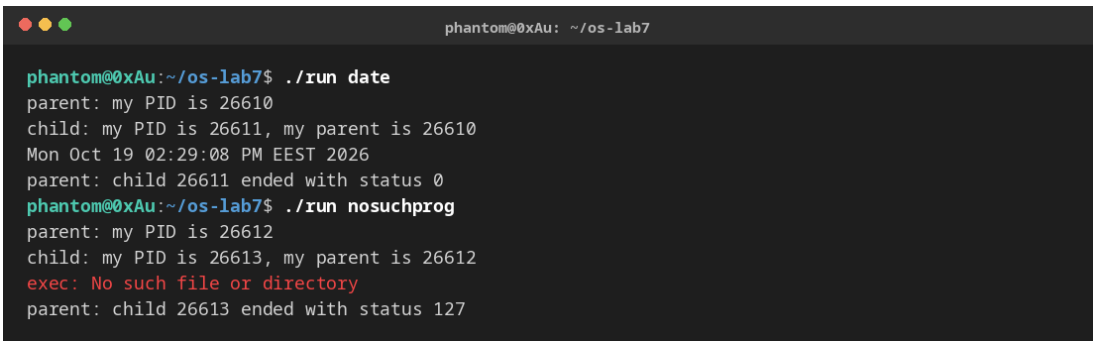


Figure 2.2 – Execution variations with date and nosuchprog

As illustrated in Figure 2.2, executing `date` resulted in normal program replacement and exit status 0. Conversely, when `nosuchprog` was supplied, the `execlp()` system call failed because the binary could not be located in any directory specified by the `$PATH` environment variable. Crucially, because `execlp()` failed, control flow reached the subsequent line in the child code, executing `perror("exec")` and emitting the error message `No such file or directory`. The child then terminated with status 127 via `exit(127)`, adhering to standard POSIX conventions.

2.2 Zombie process creation, observation, and kernel reaping

The second experimental phase examined the generation of defunct process states through the implementation of `zombie.c`. In this scenario, the parent process forked a child running `ls`, but intentionally delayed its synchronization by executing `sleep(20)` immediately prior to calling `waitpid()`. The program was executed in the background, and the kernel process table was queried using `ps -o pid,ppid,stat,cmd`.

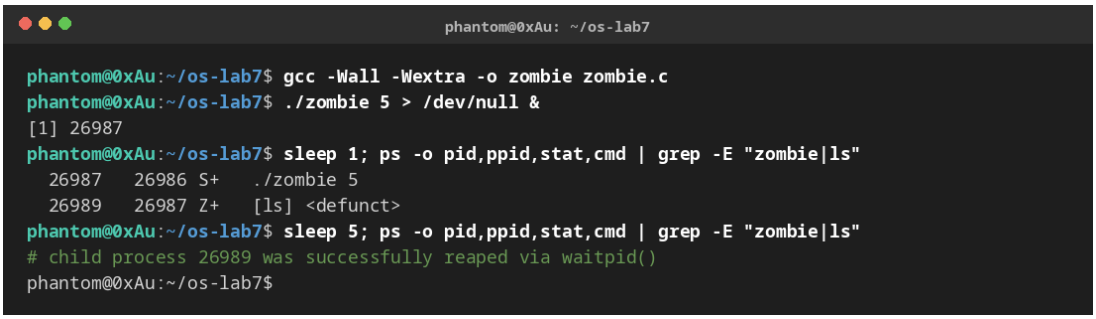


Figure 2.3 – Zombie process generation and reaping

As shown in Figure 2.3, the process table query captured the child process (PID 26989) with status state `Z+` and command identifier `[ls] <defunct>`. The state letter `Z` denotes a zombie process: the process had already executed to completion and released its memory, but its task descriptor remained allocated in kernel space awaiting status collection by parent process 26987. Once the 20-second sleep duration expired,

the parent executed `waitpid()`, reaping the child. Subsequent examination of the process table confirmed that the defunct entry was completely removed.

2.3 Standard stream redirection via deterministic descriptor allocation

The third practical phase investigated file descriptor redirection through the construction of `redirect.c`. The program emulates the shell command `ls -l > out.txt` without altering the source code of `ls`. In the child branch, the process executes `close(1)`, releasing the standard output descriptor slot. It then calls `open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644)`.

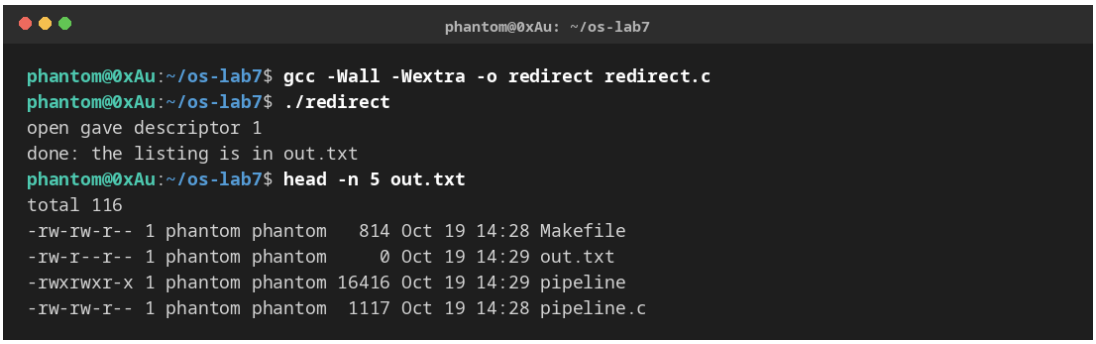


Figure 2.4 – Standard output redirection to file out.txt

According to Figure 2.4, the `open()` system call deterministically returned file descriptor 1. Because the operating system allocates the lowest available descriptor number, closing descriptor 1 ensured that `out.txt` claimed standard output. When `execlp("ls", ...)` subsequently executed, `ls` emitted its directory listing to descriptor 1 as usual. However, because descriptor 1 was bound to `out.txt`, the output stream was redirected to physical disk storage, leaving the terminal display unpopulated. Inspection with `head -n 5 out.txt` confirmed that the complete directory listing was captured in the file.

2.4 Two-stage inter-process pipeline construction and deadlock dynamics

The fourth practical phase examined inter-process communication using anonymous pipes through the implementation of `pipeline.c`, which replicates the pipeline command `ls /etc | wc -l`. The program creates a pipe via `pipe(p)` and spawns two child processes. The first child duplicates $p[1]$ into descriptor 1 using `dup()`, closes all pipe descriptors, and executes `ls /etc`. The second child duplicates $p[0]$ into descriptor 0, closes all pipe descriptors, and executes `wc -l`. The parent closes both descriptor ends and waits for both children.

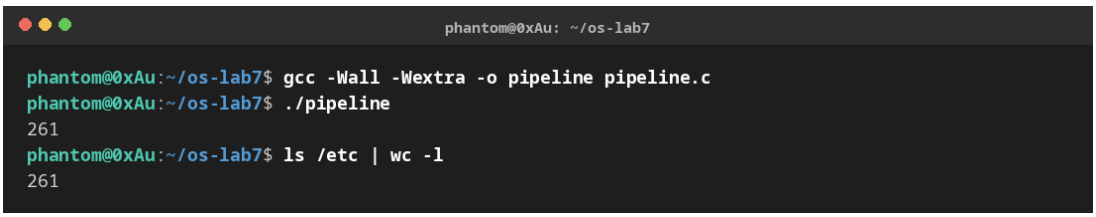


Figure 2.5 – Two-stage inter-process pipeline execution

Figure 2.5 confirms that `./pipeline` and the shell command `ls /etc | wc -l` produced the identical result of 261 lines. Data emitted by `ls` flowed through the kernel ring buffer directly into the standard input stream of `wc`.

To analyze stream synchronization and deadlock conditions, the parent program was modified to leave the write endpoint $p[1]$ open, commenting out `close(p[1])`.

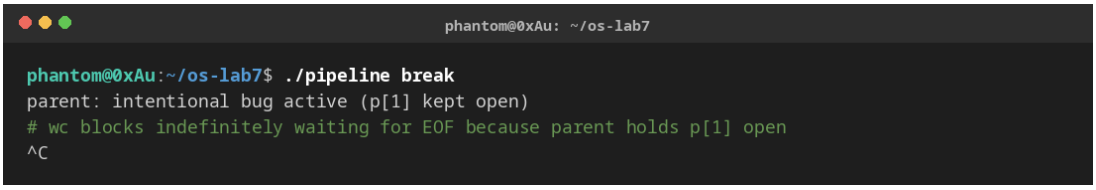


Figure 2.6 – Deadlock induction via unclosed pipe write end

As demonstrated in Figure 2.6, executing the modified pipeline caused the execution to block indefinitely. The first child (`ls`) completed and terminated normally. However, the second child (`wc`) never terminated because a pipe delivers an End-of-File (EOF) marker to the reader only when all write descriptors referencing the pipe have been closed across the entire operating system. Because the parent process maintained an open write descriptor in its own file descriptor table, the kernel could not signal EOF to `wc`, forcing it into an indefinite blocking state on descriptor 0.

2.5 Three-stage pipeline extension with multiple concurrent pipes

The optional practical extension evaluated multi-stage pipeline architectures through the implementation of `pipeline3.c`, replicating the command `ls /usr/bin | grep x | wc -l`. The architecture incorporates two distinct pipes (p_1 and p_2) and three child processes.

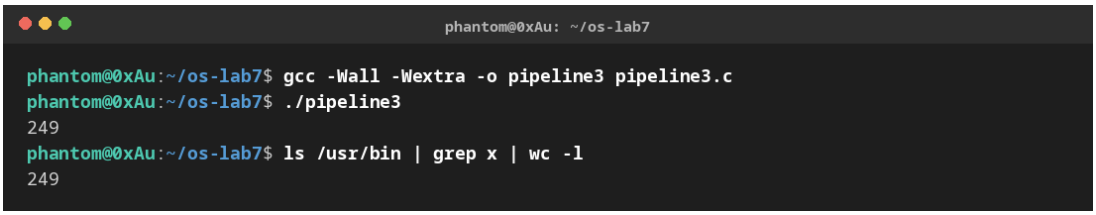


Figure 2.7 – Three-stage pipeline execution with multiple pipes

Figure 2.7 illustrates the execution of `pipeline3.c`. The first child redirects standard output to $p_1[1]$. The intermediate child binds standard input to $p_1[0]$ and standard output to $p_2[1]$. The final child binds standard input to $p_2[0]$. Each child process and the parent process rigorously close all four descriptor endpoints ($p_1[0], p_1[1], p_2[0], p_2[1]$) following duplication. Both `./pipeline3` and the shell pipeline emitted an identical count of 249 lines, validating the correctness of the multi-stage descriptor configuration.

CONCLUSIONS

This laboratory work provided a comprehensive empirical and theoretical investigation into process lifecycle management, stream redirection mechanics, and inter-process communication pipelines within the Linux operating system. By implementing low-level C utilities interacting directly with POSIX kernel system calls, the study verified the structural separation between process instantiation (`fork()`) and binary image replacement (`execve()`), demonstrating how the shell coordinates userland commands without monolithic primitives.

The practical experiments successfully demonstrated the lifecycle of zombie processes, proving that terminated child tasks retain their task descriptors in the kernel process table until explicitly harvested by parent synchronization routines (`waitpid()`). Furthermore, the examination of file descriptor redirection validated the deterministic nature of POSIX descriptor allocation, wherein closing descriptor 1 allows subsequent file allocation calls to transparently capture standard output streams without modifying target binaries.

The analysis of anonymous pipe channels highlighted critical synchronization invariants governing inter-process communication. Specifically, the intentional omission of write descriptor closure in the parent process directly induced a pipeline deadlock, illustrating that consumer utilities like `wc` rely strictly on global write endpoint exhaustion to detect EOF. Finally, the successful implementation of the three-stage pipeline (`pipeline3.c`) confirmed that complex data processing pipelines can be constructed predictably by enforcing rigorous descriptor closure disciplines across all concurrent process contexts.

REFERENCES

[1] SILBERSCHATZ, A., GALVIN, P. B., GAGNE, G. *Operating System Concepts*. 10th Edition. Hoboken: John Wiley & Sons, 2018. 1040 p. ISBN 978-1-119-32091-3.

[2] TANENBAUM, A. S., BOS, H. *Modern Operating Systems*. 4th Edition. Boston: Pearson, 2015. 1136 p. ISBN 978-0-13-359162-0.

[3] KERRISK, M. *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. San Francisco: No Starch Press, 2010. 1552 p. ISBN 978-1-59327-220-3.

[4] LOVE, R. *Linux System Programming: Talking Directly to the Kernel and C Library*. 2nd Edition. Sebastopol: O'Reilly Media, 2013. 456 p. ISBN 978-1-449-33522-9.

[5] BOVET, D. P., CESATI, M. *Understanding the Linux Kernel*. 3rd Edition. Sebastopol: O'Reilly Media, 2005. 942 p. ISBN 978-0-596-00565-8.

ANNEX A

Listing 1: Complete implementation of run.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[])
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {
        printf("child: my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        if (argc > 1) {
            execvp(argv[1], &argv[1]);
        } else {
            execlp("ls", "ls", "-l", (char *)NULL);
        }
        perror("exec");
        exit(127);
    }
    int status;
    waitpid(pid, &status, 0);
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```

Listing 2: Complete implementation of zombie.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
```

```c

int main(int argc, char *argv[])
{
    int sleep_seconds = 20;
    if (argc > 1) {
        sleep_seconds = atoi(argv[1]);
        if (sleep_seconds <= 0) sleep_seconds = 20;
    }

    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {
        printf("child: my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL);
        perror("exec");
        exit(127);
    }

    int status;
    sleep(sleep_seconds);
    waitpid(pid, &status, 0);
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```

ANNEX B

Listing 3: Complete implementation of redirect.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); exit(1); }

    if (pid == 0) {
        close(1);
        int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (fd < 0) { perror("open"); exit(1); }
        fprintf(stderr, "open gave descriptor %d\n", fd);
        execlp("ls", "ls", "-l", (char *)NULL);
        perror("exec");
        exit(127);
    }

    waitpid(pid, NULL, 0);
    printf("done: the listing is in out.txt\n");
    return 0;
}
```

Listing 4: Complete implementation of pipeline.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[])
{
```

```c
    int intentional_bug = (argc > 1 && argv[1][0] == 'b');
    int p[2];
    if (pipe(p) < 0) { perror("pipe"); return 1; }

    if (fork() == 0) {
        close(1);
        if (dup(p[1]) < 0) { perror("dup p[1]"); exit(1); }
        close(p[0]); close(p[1]);
        execlp("ls", "ls", "/etc", (char *)NULL);
        perror("exec ls");
        exit(127);
    }

    if (fork() == 0) {
        close(0);
        if (dup(p[0]) < 0) { perror("dup p[0]"); exit(1); }
        close(p[0]); close(p[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("exec wc");
        exit(127);
    }

    close(p[0]);
    if (!intentional_bug) {
        close(p[1]);
    } else {
        fprintf(stderr, "parent: intentional bug active (p[1] kept open)\n");
    }

    wait(NULL);
    wait(NULL);
    return 0;
}
```

Listing 5: Complete implementation of pipeline3.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
```

```c
{
    int p1[2], p2[2];
    if (pipe(p1) < 0) { perror("pipe p1"); return 1; }
    if (pipe(p2) < 0) { perror("pipe p2"); return 1; }

    if (fork() == 0) {
        close(1);
        if (dup(p1[1]) < 0) { perror("dup p1[1]"); exit(1); }
        close(p1[0]); close(p1[1]);
        close(p2[0]); close(p2[1]);
        execlp("ls", "ls", "/usr/bin", (char *)NULL);
        perror("exec ls");
        exit(127);
    }

    if (fork() == 0) {
        close(0);
        if (dup(p1[0]) < 0) { perror("dup p1[0]"); exit(1); }
        close(1);
        if (dup(p2[1]) < 0) { perror("dup p2[1]"); exit(1); }
        close(p1[0]); close(p1[1]);
        close(p2[0]); close(p2[1]);
        execlp("grep", "grep", "x", (char *)NULL);
        perror("exec grep");
        exit(127);
    }

    if (fork() == 0) {
        close(0);
        if (dup(p2[0]) < 0) { perror("dup p2[0]"); exit(1); }
        close(p1[0]); close(p1[1]);
        close(p2[0]); close(p2[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("exec wc");
        exit(127);
    }

    close(p1[0]); close(p1[1]);
    close(p2[0]); close(p2[1]);

    wait(NULL);
    wait(NULL);
    wait(NULL);
    return 0;
```

```
51 }
```